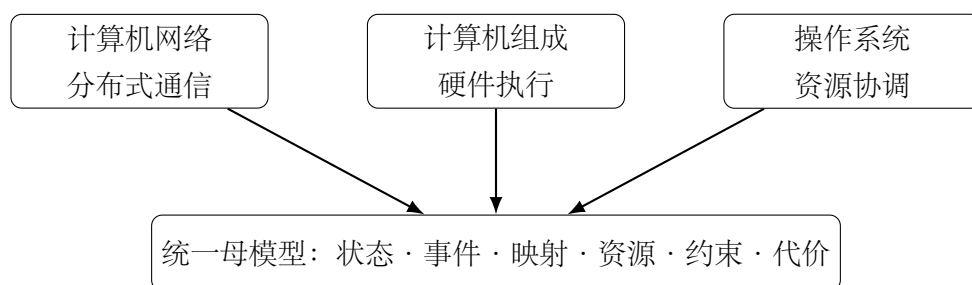


408 三科统一方法论手册

计算机网络 · 计算机组成原理 · 操作系统

从知识点记忆到计算系统状态推演



第一版 · 2026 年 8 月

面向 408 复习，同时保留通往专业系统课程的上位理解

使用说明：这不是三本教材的拼接

传统复习往往把三门课拆成大量孤立名词：TCP、页表、Cache、信号量、DMA、路由、流水线、inode……结果是“知识点都见过，综合题仍然不会推”。本手册换一个方向：**先建立系统运行的母模型，再把知识点挂回母模型。**

三门课共同研究的其实是同一个问题

一个计算系统拥有有限的物理资源，内部保存某种状态；外部或内部事件不断到来；系统依靠映射、控制规则和策略改变状态，同时必须维持正确性，并尽量降低时间、空间、带宽和可靠性成本。

$$\text{State} + \text{Event} + \text{Rule} \longrightarrow \text{New State} + \text{Cost}$$

本手册采用两层结构：

1. **主干层**：理解“为什么必须有这个机制”，训练陌生综合题的推演能力；
2. **覆盖层**：保留 408 常见知识清单和用户原始材料中的重要内容，避免为了追求“高层思想”而漏掉考试细节。

对于现代工程中已经变化、但考研教材仍采用经典模型的内容，本手册单独放入“教材模型 vs 现代工程”框，不用现代实现覆盖考试模型。

目录

使用说明：这不是三本教材的拼接	ii
第一章 统一底座：如何理解一个计算系统	1
1.1 八个跨课程母问题	1
1.2 三门课各自的母模型	2
1.3 十个反复出现的系统思想	2
1.4 三道贯穿全课程的超级母题	2
第二章 计算机网络：在没有全局即时状态的世界中通信	3
2.1 第 0 章：为什么互联网必然长成今天这样	3
2.2 五层体系真正是在划分“作用域”	3
2.2.1 封装的本质	4
2.3 网络必须掌握的十二个母思想	4
2.3.1 Scope：字段是谁看的？在多大范围内有效？	4
2.3.2 State Ownership：状态到底保存在谁手里	4
2.3.3 End-to-End：可靠性为什么常由端点最终负责	4
2.3.4 协议 = 状态 + 消息 + 定时器	5
2.3.5 统计复用：分组交换为何高效又为何会拥塞	5
2.3.6 可靠性、流量控制、拥塞控制必须三分	5
2.3.7 Data Plane / Control Plane	5
2.3.8 Naming / Addressing / Routing	5
2.3.9 复用与分用贯穿多层	6
2.3.10 Layering = 隔离变化	6
2.3.11 性能必须用时间—空间观理解	6
2.3.12 Bottleneck：端到端性能受最窄处限制	6
2.4 五层知识树：把知识点挂回母思想	6
2.4.1 应用层：业务语义与命名	6
2.4.2 运输层：端点、可靠性与反馈	7
2.4.3 网络层：跨网络寻址与分布式路径知识	7
2.4.4 数据链路层：当前一跳的帧与共享介质	8

2.4.5	物理层：信号与物理上限	8
2.5	408 高频结构补充：把“字段题”重新挂回机制	9
2.5.1	TCP 首部不应按字段表死背	9
2.5.2	IP 与 Ethernet 首部的层次差异	9
2.5.3	路由表题的固定算法	9
2.5.4	差错与可靠性：检测、纠错、重传三种路线	9
2.6	超级母题：一个 Packet 的一生	9
2.7	408 网络统一解题协议	10
第三章	计算机组成原理：把 ISA 语义变成真实硬件时序	11
3.1	第 0 章：计组真正研究什么	11
3.2	ISA 与微架构：What 与 How	11
3.3	十个计组母思想	11
3.3.1	指令即状态转移	11
3.3.2	Data Movement：真正困难的常常是“数据在哪里”	12
3.3.3	Datapath / Control	12
3.3.4	同步状态机与关键路径	12
3.3.5	单周期 / 多周期 / 流水线 = 工作如何沿时间切分	12
3.3.6	Hazard = 需要的信息/资源尚未就绪	12
3.3.7	局部性与层次	12
3.3.8	预测与投机	12
3.3.9	Resource Sharing	12
3.3.10	Performance = Compute + Move + Wait	13
3.4	数据表示：有限位宽下的语义	13
3.4.1	整数与补码	13
3.4.2	ALU 与位操作	13
3.4.3	浮点数：有限位宽下对“范围”和“精度”的重新分配	13
3.4.4	乘除、移位与溢出判断	13
3.5	指令系统：机器能做什么	13
3.6	主存组织：不仅要知道“DRAM 慢”，还要会从芯片到地址空间	14
3.7	数据通路：一条指令怎样变成微操作	14
3.8	单周期、多周期与流水线	14
3.8.1	单总线多周期	14
3.8.2	单周期	14
3.8.3	多周期	14
3.8.4	五级流水线	15
3.9	存储系统：Cache 不是表格题，而是副本管理	15

3.9.1 控制器的两类经典实现	15
3.10 总线与 I/O: 同步核心怎样接入外部世界	15
3.11 性能: 不要被单个指标骗	16
3.12 超级母题: 一条 LOAD 指令的一生	16
3.13 408 计组统一解题协议	17
第四章 操作系统: 把有限硬件变成可共享、可保护、可恢复的世界	18
4.1 第 0 章: OS 五个根本任务	18
4.2 微观轴: 控制权是怎样进入/离开内核的 (Limited Direct Execution)	18
4.3 纵轴: 一个进程的一生 (Life Cycle of a Process)	21
4.4 篇 II: 并发——所有允许的交错都必须正确	25
4.4.1 同步只有两类基本任务	25
4.4.2 锁: 不是布尔值, 而是一个小型调度系统	25
4.4.3 条件变量: 等待的是状态谓词, 不是 signal	25
4.4.4 信号量: 整数代表“许可数量”	25
4.4.5 并发错误与进展	25
4.4.6 Safety / Liveness / Fairness / Performance	26
4.4.7 经典同步模型的统一解释	26
4.4.8 事件驱动: 并发不只有多线程	26
4.4.9 Priority Inversion: 优先级与资源依赖会相互作用	26
4.5 篇 III: 内存虚拟化——一块物理内存怎样像很多私人地址空间	26
4.5.1 核心目标: 透明、效率、保护	27
4.5.2 连续分配、分段、分页: 三种不同的“空间切法”	27
4.5.3 间接映射	27
4.5.4 TLB Miss、Page Fault、Cache Miss 三分	27
4.5.5 快路径与异常慢路径	27
4.5.6 页面置换与局部性	28
4.5.7 延迟执行思想	28
4.6 篇 IV: I/O——同步 CPU 怎样接入异步慢设备	28
4.6.1 Buffer / Cache / Spooling 不同目的	28
4.6.2 磁盘调度必须先有设备成本模型	28
4.6.3 经典磁盘调度策略的目标差异	29
4.7 篇 V: 文件与持久化——从磁盘块到可命名对象	29
4.7.1 名称不是对象	29
4.7.2 文件系统是一组持久化映射	29
4.7.3 一次系统调用要展开成访问路径	29
4.7.4 文件访问、分配与空闲空间: 三套问题不能混	29

4.7.5 持久化不是“调用 write 就结束”	30
4.8 OS 的研究级扩展：保留思想，不挤占 408 主线	30
4.9 Protection：不是一章，而是横向约束	30
4.10 十二个 OS 横向母思想	31
4.11 超级母题：一次 read() 的一生	31
4.12 第二母题：一个进程的一生	31
4.13 408 OS 统一解题协议	32
第五章 三科统一：真正的 408 系统方法论	33
5.1 同一个思想在三门课里的不同化身	33
5.2 一条“程序读取网络数据”的跨三科路径	33
5.3 真正的综合题解题顺序	34
5.3.1 第一步：先定边界，不急着算	34
5.3.2 第二步：画动态轨迹	34
5.3.3 第三步：明确“不变量”	34
5.3.4 第四步：最后才进入数字计算	34
5.4 三科的训练记录模板	34
5.4.1 网络题记录	34
5.4.2 计组题记录	35
5.4.3 OS 题记录	35
第六章 完整查漏清单：防止“理解很高，但考试漏点”	36
6.1 计算机网络覆盖清单	36
6.2 计算机组成原理覆盖清单	36
6.3 操作系统覆盖清单	37
第七章 参考资料与边界说明	38
7.1 本手册的资料层次	38
7.2 推荐外部资料	38
7.3 最后的学习原则	38

统一底座：如何理解一个计算系统

1.1 八个跨课程母问题

面对任意系统概念，先不要问“定义是什么”，而依次问：

母问题	真正要问的核心内容
对象 Object	• 系统中正在被处理的核心实体：分组、指令、页、进程、文件、帧、Cache 块 • 实体的生命周期与控制块结构
状态 State	• 当前系统必须记住哪些控制信息？ • 谁保存这些状态？（硬件寄存器、PCB、TCB、Socket、页表项）
作用域 Scope	• 信息在什么范围内有效？ • 单跳 / 端到端 / 一个进程 / 一个地址空间 / 整个机器
映射 Mapping	• 从上层抽象到下层对象经过哪些间接层？ • 哪些编号保持、哪些改变？（逻辑页 → 物理页、域名 → IP、MAC → 端口）
资源 Resource	• 谁在竞争 CPU、内存、链路带宽、端口、总线、磁盘块或 Cache 空间？
事件 Event	• 什么事情触发状态变化？ • 时钟、中断、ACK、缺页、Cache miss、系统调用
不变量 Invariant	• 无论怎样并发调度或突发失败，哪些正确性条件必须一直成立？
代价 Cost	• 时间、空间、吞吐、等待、带宽、I/O、能耗、公平或可靠性代价是什么？

1.2 三门课各自的母模型

计算机网络

Scope → State → Event → Transition → Feedback → Cost

研究多个自治设备如何通过消息交换形成端到端通信。

计算机组成原理

State → Data → Path → Control → Timing → Cost

研究 ISA 的抽象语义怎样成为真实硬件中受时钟约束的数据流动。

操作系统

Resource → Abstraction → State → Event → Mechanism → Policy → Invariant → Cost

研究内核怎样把有限硬件虚拟化并安全地分给多个程序，同时处理并发、I/O 与故障。

1.3 十个反复出现的系统思想

1. 抽象与分层：隐藏下层复杂度，使上层只依赖稳定接口。
2. 间接层与映射：多一次查找，换来重定位、共享、隔离和灵活管理。
3. 时间复用与空间复用：有限资源在不同主体之间分时间或分空间。
4. 机制与策略：能怎样做与应该选谁/选什么严格区分。
5. 局部性与缓存：利用访问可预测性，用小而快的副本隐藏大而慢的资源。
6. 快路径与慢路径：高频正常情况极简，低频异常情况交给复杂处理。
7. 反馈与闭环控制：发送、调度、重传、拥塞等常依赖反馈而不是全局真相。
8. 并发与竞争：多个执行主体同时争用有限资源，正确性和性能都由竞争模式决定。
9. 安全性与恢复：系统不仅要“平时正确”，还要在异常、丢包、崩溃后回到允许状态。
10. 瓶颈与工作负载：不存在脱离 workload 的绝对“最优机制”，优化必须对准瓶颈。

1.4 三道贯穿全课程的超级母题

- 网络：一个 Packet 的一生；
- 计组：一条 LOAD 指令的一生；
- OS：一次 read() 的一生。

复习时，应不断把新知识挂回这三条动态路径，而不是只放在静态章节目录里。

计算机网络：在没有全局即时状态的世界中通信

2.1 第 0 章：为什么互联网必然长成今天这样

总定义

计算机网络不是“一根很长的管道”，而是大量自治设备依据局部状态，对到来的消息做局部决策，从而共同构造端到端通信的分布式系统。

网络的最终服务对象是不同主机上的**应用进程**。为了让这些进程交换数据，必须面对六个基本矛盾：

- 1. **距离**：信号传播需要非零时间，产生传播时延与 RTT；
- 2. **异构**：Wi-Fi、以太网、光纤等链路不同，需要统一的网络层抽象；
- 3. **共享**：多个流竞争链路，产生排队、丢包与拥塞；
- 4. **不可靠**：分组可能损坏、丢失、重复、乱序，需要检测、反馈和恢复；
- 5. **规模**：互联网不能让每台路由器为每台主机保存独立精确路径，需要层次寻址、CIDR、聚合和自治系统；
- 6. **无全局即时状态**：拓扑和负载信息传播也要时间，路由本身就是分布式收敛问题。

2.2 五层体系真正是在划分“作用域”

层	主要交付范围	典型 PDU	核心问题与关键动作
应用层	应用 应用	Message • 交互顺序与系统服务接口	• 双方通信语义与消息格式
运输层	进程 进程	TCP Segment / UDP Datagram • 可靠性、按序到达、流控与拥塞控制	• 交付给哪个具体通信端点（端口）
网络层	跨网络端到端	IP Datagram	• 目标位于何处？（逻辑 IP 寻址）

			• 路径选择与下一跳 路由器转发
链路层	一跳(Direct Link)	Frame	• 交给哪台相邻硬件设备? (MAC 寻址) • 组帧、差错检测与介质访问控制
物理层	一段物理介质	Bit / Symbol	• 比特流与物理信号 (电压/光/电磁波) 转换

可以把纵向交付链压成：

语义 → 端点 → 路径 → 下一跳 → 信号

2.2.1 封装的本质

“每层加一个首部”只是结果。更深的解释是：每向下一层走一步，就补充该层完成局部决策所必需的控制信息。

应用消息 → 端口/序号 → IP 地址/TTL → MAC/FCS → 信号

接收端按相反方向解封装。

2.3 网络必须掌握的十二个母思想

2.3.1 Scope：字段是谁看的？在多大范围内有效？

这是 408 网络综合题最有效的纠错框架之一。

- MAC：主要服务当前链路；
- IP：跨网络逻辑寻址；
- Port：端系统内的运输层分用；
- TTL：逐跳递减；
- TCP Seq/ACK：端系统维护的端到端状态。

做题时固定问：谁读取？谁修改？经过路由器后保持还是变化？

2.3.2 State Ownership：状态到底保存在谁手里

TCP 的连接状态、序号、接收窗口等主要位于两个端系统；普通 IP 路由器不为每条 TCP 连接保存传输状态。交换机保存 MAC 学习表，路由器保存转发表，DNS 解析器保存缓存。“协议存在”不等于“所有设备都理解并保存该协议状态”。

2.3.3 End-to-End：可靠性为什么常由端点最终负责

一跳可靠只能说明数据到达下一设备，不能证明业务最终正确。因此链路层可靠与 TCP 端到端可靠不是重复：作用域不同。端点才拥有完整的应用语义和最终正确性判断条件。

2.3.4 协议 = 状态 + 消息 + 定时器

统一写成：

Current State + Event → Action + New State

TCP、DHCP、ARP、路由协议都可以按这个模型推演。时间尤其重要，因为发送方不能瞬间知道远端发生了什么，只能通过 ACK、超时和重传推断远端状态。

2.3.5 统计复用：分组交换为何高效又为何会拥塞

多个用户按需共享链路提高利用率，但当到达速率长期超过服务速率时，队列增长、时延升高、最终丢包。因此拥塞是统计复用的自然代价，而不是 TCP “制造”的问题。

2.3.6 可靠性、流量控制、拥塞控制必须三分

机制	保护对象	核心目标与主要约束参数
可靠性 Reliability	通信语义	- 保证数据无差错、无丢失、无重复、按序到达 - 依赖 ACK、校验和、超时重传与滑动窗口
流量控制 Flow Control	接收方	- 防止发送方太快抹平接收方缓冲区能力 - 核心约束参数为接收窗口 <i>rwnd</i>
拥塞控制 Congestion Control	网络路径	- 防止过多数据涌入中间瓶颈链路和路由器 - 核心约束参数为拥塞窗口 <i>cwnd</i>

发送方可保留的在途数据通常受二者较小值限制：

$$W_{send} \leq \min(rwnd, cwnd).$$

2.3.7 Data Plane / Control Plane

数据面回答“这个包现在来了，怎么转”；控制面回答“转发表从哪里来”。最长前缀匹配属于转发；RIP/OSPF/BGP 等负责产生和更新可达性知识。

2.3.8 Naming / Addressing / Routing

Name → Address → Route → Next-hop Link Address

DNS 主要做域名到 IP 资源记录的解析；IP 转发用目的 IP 选择下一跳；IPv4 本地链路常再用 ARP 把下一跳 IP 解析为硬件地址。

2.3.9 复用与分用贯穿多层

运输层用端口复用多个应用；网络和链路让多个流共享链路；物理层可用 TDM/FDM/WDM/CDM 等共享介质。网络反复在做“Many $\rightarrow$ shared resource $\rightarrow$ Many”。

2.3.10 Layering = 隔离变化

分层的价值不是分类，而是让 HTTP 的变化不必迫使物理链路重做，让 Wi-Fi 与以太网变化不必重写应用。但分层是分析模型，不是不可跨越的自然定律，NAT、代理、VPN、QUIC 等都会跨越传统边界。

2.3.11 性能必须用时间—空间观理解

发送时延：

$$d_{trans} = \frac{L}{R}$$

是把 L 比特串行送入链路所需时间；传播时延：

$$d_{prop} = \frac{D}{v}$$

是信号跨越物理距离所需时间。二者物理含义不同。

带宽时延积：

$$BDP \approx Bandwidth \times Delay$$

表示“管道里能同时容纳多少在途数据”。窗口过小会让长 RTT 高带宽链路空转。

2.3.12 Bottleneck：端到端性能受最窄处限制

一条路径的长期吞吐不会超过瓶颈链路能力。提高非瓶颈部分并不自动提高整体性能。

2.4 五层知识树：把知识点挂回母思想

2.4.1 应用层：业务语义与命名

- **DNS**：层次命名、根/TLD/权威服务器、递归解析与 TTL 缓存；传统查询常用 UDP 53，也可使用 TCP，不能死记“DNS 只走 UDP”。
- **HTTP**：方法、URL/资源、首部、状态码、请求/响应语义；2xx/3xx/4xx/5xx 分类。
- HTTP/1.1 与 HTTP/2 通常建立在 TCP 上；HTTP/3 把 HTTP 语义映射到 QUIC。
- **FTP**：控制连接典型 TCP 21；主动模式服务器数据连接传统使用 TCP 20，被动模式使用动态端口。
- **SMTP / POP3 / IMAP**：分别承担发送/转发、下载、服务器侧同步管理等角色。
- C/S 与 P2P：核心差异是资源与服务能力的组织方式，实际 P2P 也可能依赖协调服务器。

教材模型 vs 现代工程：HTTP/3

HTTP/3 是 HTTP 语义在 QUIC 上的映射；QUIC 自身提供多路流、流量控制、丢失恢复等运输能力并运行在 UDP 之上。408 主线仍应先掌握 HTTP/TCP 的经典教材模型，再用 HTTP/3 校正“HTTP 永远必须直接运行在 TCP 上”的绝对化说法。

2.4.2 运输层：端点、可靠性与反馈

- Port 标识运输层通信端点，不等于 OS 进程号；TCP 连接通常用源/目的 IP 与端口四元组区分，跨协议比较时可扩展为五元组。
- UDP：无连接、保留消息边界、不负责确认重传排序与拥塞控制；校验和只能检测差错的一部分，不等于可靠交付。
- TCP：面向连接、可靠有序字节流，不保留应用写入边界；应用协议需自行定义消息边界。
- 三次握手的核心是双向同步连接状态与初始序号；关闭是两个单向方向的独立结束过程，典型四段但 ACK/FIN 可合并。
- TIME_WAIT：服务于最终 ACK 的潜在重传和旧报文消散。
- 累计确认、超时重传、经典“三个重复 ACK”快速重传；SACK 可报告非连续已收区间。
- 滑动窗口提高在途数据量；GBN/SR 是教学模型，TCP 不等同于其中任一简单版本。
- 经典拥塞控制模型：慢启动、拥塞避免、快速重传、快速恢复；实际算法和参数可能演进。

2.4.3 网络层：跨网络寻址与分布式路径知识

- IP 提供 Best-Effort，不保证到达、顺序、唯一性或固定时延。
- 转发与路由选择分离；主机向本地子网外发送通常交给默认网关。
- IPv4 首部重点：源/目的地址、协议、TTL、总长度、首部校验和、分片字段、区分服务相关字段。
- TTL 按路由跳递减，应理解为生存跳数限制。
- IPv4 分片：片偏移以 8 字节为单位，除最后片外数据长度通常为 8 的整数倍，重组在最终目的主机。
- CIDR、子网划分、路由聚合、最长前缀匹配是规模化寻址的核心。
- DHCP：典型 DORA；还提供 mask、gateway、DNS、lease；续租与重新绑定状态不能用首次广播流程机械替代。经典 DHCPv4 教材模型中，未另行指定时常取 $T_1 = 0.5 \times lease$ 、 $T_2 = 0.875 \times lease$ 。
- ICMP：目的不可达、TTL 超时等；ping 常用 Echo；traceroute/tracert 利用逐步增加 TTL/Hop Limit 的返回信息推测路径。
- NAT 是地址转换机制族；NAPT 用地址 + 端口映射多个内部连接，会引入中间状态并削弱纯粹端到端可达性。
- IPv6：128 位地址、固定 40 字节基本首部、扩展首部、无 IPv4 式广播；中间路由器不执行 IPv6 分片；连续零可用 :: 压缩，但一个地址中只能用一次。

- 距离向量思想接近 Bellman-Ford；链路状态协议泛洪拓扑信息后运行 SPF。
- RIP：跳数度量，15 可达、16 不可达；OSPF：链路状态与区域；BGP：AS 间路径向量与策略可接受性，不只追求数学最短路径。
- IP 多播在路径分叉处复制；Mobile IP 是教材中的经典移动性机制。

教材模型 vs 现代工程：IPv6 分片

IPv6 的分片只由源节点执行，转发路径上的路由器不做 IPv4 式中途分片。因此应把“IPv4 分片规则”和“IPv6 源端分片”严格分开。

2.4.4 数据链路层：当前一跳的帧与共享介质

- 组帧与透明传输：字符计数、字节填充、比特填充、违规编码；HDLC 标志 01111110，连续五个 1 后插 0。
- 差错检测：奇偶、CRC；FEC 如海明码用于纠错。CRC 检测不等于自动恢复。
- PPP：点到点封装和 FCS，通常不提供可靠重传。
- MAC 解决共享介质接入；FDM/TDM/WDM/CDM 属于信道划分，ALOHA/CSMA 属于随机接入。
- CSMA/CD：共享介质半双工以太网，最小帧长与最坏往返传播时间相关；现代交换式全双工以太网通常不存在此类碰撞。
- CSMA/CA：802.11 使用监听、随机退避、确认；RTS/CTS 可缓解隐藏终端，但不是每帧必用。
- MAC 地址常见 48 位；软件设置、虚拟化等意味着不能把“全球永不重复”当成绝对物理定律。
- ARP：IPv4 下一跳 IP 到本地硬件地址；远端目的主机不在本子网时解析的是网关 MAC；IPv6 使用 NDP 而非 ARP。
- Hub 属物理层；Bridge/Switch 按 MAC 转发。交换机从源 MAC 学习端口；未知单播/广播等在 VLAN 内可能泛洪。
- 交换机隔离冲突域，不天然隔离广播域；VLAN/三层路由划分广播域。

2.4.5 物理层：信号与物理上限

- 信源、信宿、信道、信号；比特率与码元率严格区分。
- 若每码元有 M 个可区分状态，则每码元信息量 $\log_2 M$ ，有 $R_b = R_s \log_2 M$ 。
- 编码与调制：数字数据到数字信号、载波幅频相变化等。
- Nyquist 理想无噪声带宽限制： $C = 2B \log_2 M$ （教材常用形式）。
- Shannon 有噪声容量： $C = B \log_2(1 + S/N)$ ； $SNR_{dB} = 10 \log_{10}(S/N)$ 。
- 介质：双绞线、同轴、光纤、无线；中继器再生信号，Hub 是多端口中继器。
- 电路交换、报文交换、分组交换；分组交换支持统计复用但带来排队、抖动和丢包。

- 数据报网络逐包独立转发；虚电路网络在中间节点保留逻辑路径状态。TCP 连接状态主要在端系统，不能把 TCP 当成网络层虚电路。

2.5 408 高频结构补充：把“字段题”重新挂回机制

2.5.1 TCP 首部不应按字段表死背

源/目的端口解决分用；Seq/ACK 维护可靠字节流；Window 服务流量控制；Checksum 检测传输差错；SYN/FIN/RST/ACK 等控制位驱动连接状态机。做字段题时固定问：这个字段改变哪一个状态变量？

2.5.2 IP 与 Ethernet 首部的层次差异

IP 目的地址服务跨网络转发；Ethernet 目的 MAC 服务当前链路。一台路由器转发数据报时通常拆掉旧二层帧、更新网络层逐跳字段（如 TTL 与相应首部校验）、选择下一跳，再使用新链路的二层首部封装。没有 NAT 等中间机制时，端到端源/目的 IP 的语义通常保持。

2.5.3 路由表题的固定算法

1. 将目的地址与候选前缀比较；
2. 找出所有匹配表项；
3. 选择前缀长度最大的表项（Longest Prefix Match）；
4. 得到下一跳/输出接口；
5. 若为直连网络，下一跳就是目的主机本身；否则是路由器。

2.5.4 差错与可靠性：检测、纠错、重传三种路线

CRC/Checksum 主要回答“能否发现错误”；FEC 通过冗余尝试在不重传时恢复；ARQ/TCP 重传则通过反馈重新发送。三者的冗余位置与适用场景不同。

2.6 超级母题：一个 Packet 的一生

假设一台新主机第一次访问一个远程 HTTPS 网站：

接入网络 → DHCP 获得 IP/Mask/Gateway/DNS → DNS: Name→IP → 判断是否同子网 → 确定 Next Hop → ARP/NDP 获取链路地址 → 封装 Frame[IP[TCP/QUIC[应用数据]]] → Switch 按 MAC → Router 解旧帧、TTL--、LPM、重新封装 → 目的主机运输层分用 → 应用处理 → 响应反向返回

这条链必须能回答：

1. 每一步“谁知道什么”？
2. 哪些字段端到端保持，哪些逐跳变化？

3. 当前失败会触发什么反馈或超时?
4. 当前资源瓶颈和主要时延来自哪里?

2.7 408 网络统一解题协议

408 训练：网络七问

1. 当前对象是什么：bit/frame/datagram/segment/message?
2. 当前作用域是一跳、端到端还是某个 AS/子网?
3. 哪些状态由谁维护?
4. 发生了什么事：发送、收到 ACK、超时、查表、碰撞、TTL 到零?
5. 哪个字段决定下一步，谁会修改它?
6. 最终量是地址、序号、窗口、时延、吞吐还是路径？单位是否正确?
7. 这是正常路径还是异常/重传/分片/拥塞慢路径?

网络高频混淆

带宽 $\neq$ 吞吐；发送时延 $\neq$ 传播时延；MAC $\neq$ IP；端口 $\neq$ PID；转发 $\neq$ 路由；流量控制 $\neq$ 拥塞控制；DNS 不等于只用 UDP；HTTP 不等于永远直接使用 TCP；TCP 连接 $\neq$ 网络层虚电路；交换机隔离冲突域 $\neq$ 自动隔离广播域。

计算机组成原理：把 ISA 语义变成真实硬件时序

3.1 第 0 章：计组真正研究什么

总定义

计算机组成原理研究：如何用有限数量、有限速度、有限带宽的物理硬件，正确而高效地实现 ISA 对软件承诺的程序语义。

一条指令可抽象成体系结构状态转移：

$$S_{t+1} = F_I(S_t)$$

其中 S 包含 PC、体系结构寄存器、可见内存状态、标志/控制状态等。计组所有优化的硬约束是：内部可以更复杂、更重叠，外部观察到的程序语义不能被破坏。

3.2 ISA 与微架构：What 与 How

抽象层次	定义与核心回答问题
ISA (体系结构)	• 软件可见硬件接口：指令集、寄存器、数据类型、寻址方式、异常/特权机制 • 回答“机器向软件承诺提供什么”
微架构 (Microarchitecture)	• 内部物理实现：数据通路、控制器、流水线、Cache、分支预测、内部微操作 (μops) • 回答“内部如何高效完成上述硬件承诺”

同一 ISA 可以有多种处理器实现；现代 x86 常把复杂指令解码成内部 μops ，而软件仍看到同一 ISA。RISC/CISC 是设计倾向，不是完全对立的两个盒子。

3.3 十个计组母思想

3.3.1 指令即状态转移

指令类型虽然很多，但都可以看成对体系结构状态的合法修改：算术修改寄存器，LOAD/STORE 在寄存器与存储层次间移动数据，branch/jump 修改 PC，系统控制指令影响特权状态。

3.3.2 Data Movement：真正困难的常常是“数据在哪里”

计算 $C = A + B$ 之前必须解决：A/B 在寄存器、Cache、DRAM 还是设备？怎样寻址？走哪条总线？ALU 输入从哪个 MUX 选？结果写到哪里？因此计组大量设计是在**组织数据移动**，不是在重新发明加法。

3.3.3 Datapath / Control

Datapath 回答“有哪些路可以走”；Control 回答“此刻打开哪些路”。控制器根据 opcode、时序状态、条件标志和异常等产生寄存器写使能、ALU 操作、存储器读写、MUX 选择等信号。

3.3.4 同步状态机与关键路径

典型同步数字系统：

$Register \rightarrow Combinational\ Logic \rightarrow Register$

时钟周期必须容纳最坏关键路径及必要时序裕量。因此频率并非任意设定，而受最慢组合逻辑约束。

3.3.5 单周期 / 多周期 / 流水线 = 工作如何沿时间切分

单周期把整条最长指令路径塞进一个周期；多周期把工作拆开并复用硬件；流水线进一步让不同指令占据不同阶段并重叠。三者本质是在回答：**工作怎样切分，硬件怎样复用，吞吐怎样提高。**

3.3.6 Hazard = 需要的信息/资源尚未就绪

结构冒险：资源未就绪；数据冒险：值未就绪；控制冒险：下一 PC 未就绪。解决方案可统一为：**等 (stall) / 绕 (forwarding) / 加资源 / 猜 (prediction)。**

3.3.7 局部性与层次

CPU 与主存速度、容量、成本无法同时最优，于是：

$$Register \rightarrow L1 \rightarrow L2 \rightarrow L3 \rightarrow DRAM \rightarrow SSD/HDD.$$

Cache 利用时间/空间局部性，用小而快的副本隐藏大而慢的状态。

3.3.8 预测与投机

分支方向未知时，保守等待浪费流水线。预测器用历史估计未来，猜对获得性能，猜错支付 flush/recovery 成本。高性能设计不断在“平均收益 vs 错误代价”之间取舍。

3.3.9 Resource Sharing

总线、ALU、Memory port、Cache 组、DMA 通道等都是有限共享资源。结构冒险、总线仲裁、替換算法本质都是“谁在何时占用有限资源”。

3.3.10 Performance = Compute + Move + Wait

经典公式：

$$T_{CPU} = IC \times CPI \times T_{cycle} = \frac{IC \times CPI}{f}.$$

更高层看，程序时间由**有用计算**、**数据移动**、**等待停顿**组成。优化必须对准瓶颈，不能只看频率、IPC 或 MIPS 单一指标。

3.4 数据表示：有限位宽下的语义

3.4.1 整数与补码

n 位无符号范围：

$$0 \leq x \leq 2^n - 1.$$

n 位补码范围：

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

补码把减法统一到模 2^n 的加法框架，零只有一种编码，符号扩展自然。实际硬件减法仍需反相/进位控制和标志逻辑，不能粗暴说“CPU 完全不需要减法相关电路”。

3.4.2 ALU 与位操作

ALU 执行算术、逻辑、比较和移位；按位与/或/异或/取反用于掩码、权限字段、编码与地址处理。数据表示决定 ALU 如何解释同一位模式以及怎样判断进位、溢出和符号。

3.4.3 浮点数：有限位宽下对“范围”和“精度”的重新分配

408 中浮点数应至少掌握符号位、阶码、尾数/有效数的角色，以及 IEEE 754 单精度/双精度的基本组织思想。与定点整数相比，浮点用指数扩大动态范围，但牺牲均匀精度；规格化、舍入、溢出/下溢和特殊值都来自“有限位模式近似实数”的根本约束。做题时不要把二进制位串直接当实数，必须先按字段解释。

3.4.4 乘除、移位与溢出判断

补码加减只是数据运算的一部分。408 还常考乘法/除法的移位加减思想、算术/逻辑移位区别、无符号进位与有符号溢出的不同判定。核心仍是：**同一位模式在不同数据语义下，标志位含义不同**。

3.5 指令系统：机器能做什么

- 指令由 Opcode 与操作数/寻址信息构成；地址字段可能是寄存器号、立即数、偏移或地址计算信息，不一定是完整内存地址。
- 指令类型：算术、逻辑、移位、数据传送、访存、分支/跳转、调用/返回、系统控制。
- 高级语句一般对应多条机器指令；汇编助记符是可读表示，具体编码由 ISA 决定。

- MIPS、ARM、RISC-V 常作为 RISC 代表；x86 常作为 CISC 代表。现代实现相互吸收设计思想。

教材模型 vs 现代工程：RISC-V 作为“接口/实现分离”的实例

RISC-V 官方规范明确区分 unprivileged ISA 与 privileged architecture；特权体系可以在不改变非特权 ISA 的情况下采用不同设计。这非常适合用来理解“软件可见 ISA 与 OS/硬件特权机制相互分层”。

3.6 主存组织：不仅要知道“DRAM 慢”，还要会从芯片到地址空间

408 的存储题还需要把“抽象容量”落到实际芯片组织：地址线决定可选单元数量，数据线决定每次并行读写位数；位扩展解决字长不足，字扩展解决字数不足，片选信号决定当前激活哪组芯片。多模块存储器/低位交叉编址等设计本质上是用**并行 bank**提高连续访问带宽。

SRAM/DRAM 的差异不只用于背 Cache/主存用途，还应联系刷新、速度、密度和成本。ROM/Flash 等非易失存储则用于不同持久性需求。所有这些仍可归到“容量—速度—成本—持久性”的层次权衡。

3.7 数据通路：一条指令怎样变成微操作

经典处理步骤可写成：

Fetch → Decode/RegisterRead → Execute → Memory → WriteBack.

教材中的“取指周期、间址周期、执行周期、中断周期”是指令周期的另一种划分；两套分类所处抽象层不同，不应机械混同。

关键部件：PC、IR、GPR、ALU、Control Unit、Memory Interface、Datapath。控制器不是简单“把指令翻译成电信号”，而是根据指令和机器状态生成控制选择。

3.8 单周期、多周期与流水线

3.8.1 单总线多周期

共享内部通路节省硬件，但同一时刻可进行的数据传送有限，需要把指令拆成多个微操作周期；总线容易成为资源冲突点。

3.8.2 单周期

理论 CPI=1，但时钟周期必须覆盖最慢指令完整路径，简单指令被最长路径拖慢。它适合教学与简单实现，不代表现代通用高性能 CPU 的主流微架构。

3.8.3 多周期

不同指令使用不同周期数，可复用 ALU/Memory 接口并缩短单周期关键路径。比较性能不能只看 CPI，还要结合周期时间。

3.8.4 五级流水线

IF/ID/EX/MEM/WB 只是经典模型；理想填满后 IPC 可接近 1，但单条指令仍经过多级。流水改善吞吐而不保证降低单条 latency。

三类 hazard 与常见解决：

- Structural：复制/分离硬件、stall；
- Data：forwarding、stall、编译器调度；
- Control：branch prediction、提前判定、错误路径 flush。

3.9 存储系统：Cache 不是表格题，而是副本管理

Cache 的四个统一问题：

1. Placement：主存块可以放哪里？直接/组相联/全相联；
2. Identification：如何确认当前行是不是目标块？Tag；
3. Replacement：满了牺牲谁？
4. Write：副本被修改后何时更新下一层？write-through/write-back，配合 write-allocate/no-write-allocate。

地址常分为：

Tag	Index	Offset
-----	-------	--------

分别回答“哪一个主存块 / 去哪组 / 块内哪一字节”。

SRAM 常用于 Cache，DRAM 常用于主存；寄存器、Cache、主存、辅存构成速度—容量—成本层次。多个缓存副本还会引入一致性问题，尤其在多核系统中。

3.9.1 控制器的两类经典实现

硬布线控制把控制逻辑直接做成组合/时序电路，通常速度快但修改困难；微程序控制把微操作控制序列编码进控制存储器，设计规整、扩展方便但多一层微指令解释。二者都在实现同一个映射：指令与当前状态 → 控制信号序列。

3.10 总线与 I/O：同步核心怎样接入外部世界

Bus 不是“几根线”，而是：

<i>Shared Communication Resource + Protocol</i>

需要解决寻址、数据、控制、仲裁、时序和不同速度设备协调。按传递内容可分数据/地址/控制总线；现代系统也可采用点到点互连、NoC 等，但问题模型不变。

I/O 的根本矛盾：CPU 时钟驱动且快，外设事件驱动且慢。

I/O 方式	CPU 参与度	核心思想与机制
轮询 Polling	高	• CPU 主动反复查询外设状态寄存器 • 实现简单但严重浪费 CPU 等待时间
中断 Interrupt	中	• 外设准备就绪后异步发送中断信号通知 CPU • 解除 CPU 主动轮询等待
DMA 控制	低	• CPU 仅初始化配置源/目的地址与传输长度 • DMA 控制器批量搬运数据，结束后统一中断通知

中断、异常、系统调用都可能导致控制流转移，但来源与同步性不同；CPU 需在体系结构规定的边界保存必要状态并进入处理路径。

3.11 性能：不要被单个指标骗

- Clock rate、CPI、IPC、IC 必须结合工作负载；
- $IPC \approx 1/CPI$ 只是在一致统计口径下的平均关系；乱序、多发射、空闲等会使简单直觉失真；
- 性能单位 MIPS 不适合跨 ISA/跨程序直接比较；同名 MIPS 还可能指 MIPS 指令集；
- 最可靠比较是相同任务与输入下的实际完成时间，并说明编译、功耗和硬件条件。

更高层并行方式：Superscalar、Superpipelining、VLIW、Multicore/Multiprocessor。它们受程序并行度、依赖、Cache/Memory 系统和 OS 调度限制，不能简单理解为“多放几个 CPU”。

3.12 超级母题：一条 LOAD 指令的一生

以 `LOAD R1, 8(R2)` 为例：

PC→I-Cache 取指 → Decode(opcode,R1,R2,imm) → 读 R2 → ALU 计算 EA=R2+8 → D-Cache 查找 → Hit：返回数据 / Miss：逐级访问 L2/L3/DRAM → 写回 R1 → 对 ISA 呈现新的体系结构状态

如果放进流水线，还应继续问：

1. 每一步在哪个 cycle?
2. 后续指令若立即使用 R1，值在哪一级产生，forwarding 是否足够?
3. Cache miss 会造成多少 stall?
4. 地址字段如何分 Tag/Index/Offset?
5. 最终真正提交的体系结构状态是什么?

建议配合 ADD、STORE、BRANCH 四类母指令：纯计算、数据读入、数据写出、控制流改变，几乎覆盖教学处理器核心路径。

3.13 408 计组统一解题协议

408 训练：计组六问

1. State: PC、寄存器、Cache/Memory 当前状态是什么?
2. Location: 需要的数据当前在哪里?
3. Path: 必须经过哪些部件和 MUX/总线?
4. Resource: 有没有硬件资源竞争?
5. Timing: 每个值在哪个周期产生、什么时候可用? 关键路径是什么?
6. Commit/Cost: 最终修改哪个体系结构状态, CPI/时延/命中代价是多少?

计组高频混淆

ISA $\neq$ 微架构; CPI 低 $\neq$ 程序必然快; 流水线吞吐提高 $\neq$ 单条指令变成一个周期; Cache miss $\neq$ Page Fault; 地址字段 $\neq$ 完整内存地址; 中断 $\neq$ DMA; 时钟频率高 $\neq$ 实际任务一定快; “指令和数据都是二进制” $\neq$ 所有机器都采用完全统一的物理指令/数据存储结构。

操作系统：把有限硬件变成可共享、可保护、可恢复的世界

4.1 第 0 章：OS 五个根本任务

总定义

操作系统是运行在特权级上的、事件驱动的资源虚拟化与协调系统：它把有限而复杂的硬件资源包装成稳定抽象，让多个互不信任、彼此并发的程序安全共享资源，并在异常、I/O 和故障发生时维持可解释状态。

五个根本任务：

Control + Virtualization + Coordination + Protection + Persistence

用户给出的 OS 初稿已经用“资源是什么、抽象是什么、谁竞争、状态记录在哪里、什么事件改变状态、硬件/内核分工、快慢路径、不变量和代价”等问题组织全局，这正适合作为本篇的底层检查框架。

4.2 微观轴：控制权是怎样进入/离开内核的（Limited Direct Execution）

1. 根本矛盾：为什么需要 Limited Direct Execution?

CPU 只有一个核心，OS 既要让用户代码在硬件上**原生直接执行（Direct Execution）**以获得极高性能，又必须确保用户程序**不能执行特权指令、不能非法越界访存、不能永久霸占 CPU (Limited)**。因此，OS 必须依赖硬件提供的特权级与捕获机制，受控地让出与回收 CPU 控制权。

2. 对象模型：控制权转移中的核心实体

- **用户态 CPU (User Mode)**: CPL/RPL 为 3，受限与非特权指令集与 MMU 页表权限保护。
- **内核态 CPU (Kernel Mode)**: CPL/RPL 为 0，具备执行特权指令（如修改 CR3、关中断、写 IDT）的最高权限。
- **中断描述符表 (IDT)**: 系统启动时由内核初始化在内核空间，保存中断/异常号到内核处理入口点（Trap Handler）的映射。
- **用户栈 (User Stack)**: 位于用户虚拟地址空间高位/低位，保存用户函数调用的局部变量与返回地址。

- **内核栈 (Kernel Stack)**: 每个进程在内核空间独占的独立堆栈，用于保存 Trap 发生时的硬件上下文 (Trap Frame) 与内核函数调用链。

3. 状态模型：三层状态的严格划分

状态分层	所包含的具体硬件 / 数据结构信息
用户可见状态	• 用户通用寄存器（如 EAX/RAX 等）、用户栈指针 SP、程序计数器 PC • 仅由用户程序指令正常读写与修改
CPU 硬件执行状态	• 当前特权级 (Mode Bit / CPL)、页表基址寄存器 (CR3)、中断允许位 (IF) • 硬件 Trap 发生时自动关中断或切换特权级
内核管理状态	• 进程 PCB (进程状态、PID、优先级)、内核栈 (Kernel Stack)、Trap Frame • 仅由内核态代码读写，用户态完全不可见

4. 不变量 (Invariants)

1. **入口唯一性**: 用户态进入内核态的唯一合法途径是硬件 Trap 机制，绝不能通过任意 JMP/CALL 跳转到内核代码；
2. **栈隔离性**: 处于内核态执行时，必须使用当前进程的**内核栈**，绝不能为用户栈上压入内核敏感控制信息；
3. **状态可逆性**: Trap 发生时硬件与内核保存的状态，必须足以在 `return-from-trap` 时无缝恢复用户程序的执行。

5. 事件集合：触发控制权转移的三类事件

- **System Call (系统调用)**: 用户程序主动发起（如 `read()`、`fork()`），属于**同步意图事件**；
- **Exception (异常)**: 当前 CPU 执行指令内部触发（如除零、缺页 Page Fault、非法指令），属于**同步错误/缺页事件**；
- **Interrupt (中断)**: 外部硬件设备或时钟定时器异步发送（如 Timer Interrupt、Disk Ready），属于**异步外部事件**。

6. 动态轨迹：一次系统调用 (Syscall) 从用户态到内核态的单步图景

时刻	运行主体	CPU 模式	堆栈与寄存器	触发事件	硬件与内核完成的动作
T ₀	用户程序	User Mode	SP=UserStack, PC=UserApp	调用 <code>read()</code>	压参数入用户栈，加载系统调用号到指定寄存器。

T_1	硬件 Trap	Mode Switch	硬件将 SP 切换为 KernelStack	执行 <code>syscall</code> / <code>int</code>	硬件自动保存 SS, ESP, EFLAGS, CS, EIP 到内核栈, Mode Bit 设为 0, PC 设为 <code>IDT[trap_no]</code> 。
T_2	Trap Entry	Kernel Mode	SP=KernelStack, 保存通用寄存器	进入 <code>alltraps</code>	内核汇编入口将剩余通用寄存器压入内核栈, 构造完整 <code>TrapFrame</code> 。
T_3	Trap Handler	Kernel Mode	SP=KernelStack, 执行 <code>sys_read()</code>	内核服务处理	根据系统调用号查表执行业务逻辑; 若无需阻塞, 将返回值存入 <code>TrapFrame</code> 中的 <code>EAX</code> 。
T_4	Trap Return	Kernel Mode	弹出通用寄存器, 准备恢复用户上下文	执行 <code>sysret</code> / <code>iret</code>	从内核栈恢复 EIP, CS, EFLAGS, ESP, SS, 硬件 Mode Bit 恢复为 3 (User Mode)。
T_5	用户程序	User Mode	SP=UserStack, PC=UserApp+4	系统调用返回	用户程序拿到 <code>EAX</code> 中的返回值, 继续向下执行。

7. 分支路径：控制权转移的三条演化路径

- **Fast Path (直接返回)**: 无须等待 I/O, 更新 `EAX` → 执行 `sysret` 返回原进程用户态。
- **Slow Path (阻塞/抢占)**: 发起磁盘 I/O 或被时钟中断抢占 → 触发 **Context Switch** 切换到其他进程。
- **Exception Path (严重致命)**: 越界/非法指令无法修复 → 向进程发送 `SIGSEGV` 信号或直接终止进程。

8. 机制与策略：模式切换 vs 上下文切换

408 核心概念区分：Mode Switch vs Context Switch

- **Mode Switch (模式切换)**: 同一个进程从用户态进入内核态 (或返回)。只需在内核栈保存 CPU 寄存器上下文 (Trap Frame), **进程地址空间 (CR3)、PCB 状态不发生改变**。开销小。
- **Context Switch (上下文切换)**: CPU 执行流从进程 A 彻底切换到进程 B。必须由内核调度器 (Scheduler) 触发, 不仅保存 A 的内核栈/寄存器, 还需**切换 CR3 (更新页表/刷 TLB) 并更新 PCB 状态为 Ready**。开销大。

9. 408 题型映射

408 训练：控制权转移高频考点

- 1. **中断 vs 异常判别**：时钟中断、网卡到达属于异步中断；除零、缺页、系统调用 trap 属于同步异常。
- 2. **特权指令判别**：修改 CR3、关中断、写 IDT 只能在内核态执行；取指、ADD、用户态栈操作为非特权指令。
- 3. **栈切换考题**：特权级切换时必须切换堆栈（从用户栈切到内核栈）；同一个进程在内核态执行时使用其专属内核栈。

10. 诊断与错因矩阵

学生典型错误表象	底层认知缺口诊断	正确推演矫正句
误以为中断发生就必然发生进程切换	混淆 Mode Switch 与 Context Switch	中断处理完成后，若无更高优先级或抢占，控制权直接 <code>iret</code> 返回当前被中断进程。
认为系统调用是在用户栈上执行内核代码	未掌握用户栈与内核栈的隔离保护机制	硬件 Trap 发生的第一动作就是把 SP 换为内核栈，内核代码绝不在用户栈压栈。
误以为异常是外部硬件发出的信号	混淆 CPU 指令内部触发与外部设备中断	异常是 CPU 正在执行的当前指令同步触发的；中断是外部设备发出的异步信号。

4.3 纵轴：一个进程的一生（Life Cycle of a Process）

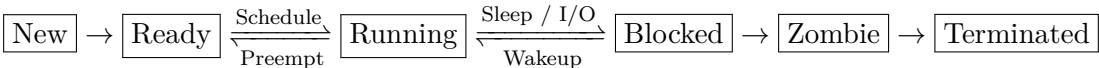
1. 根本矛盾：从静态文件到动态运行

磁盘上的程序（ELF 文件）只是静态的代码与数据字节序列。为了让程序运行，OS 必须为它建立**独立地址空间、分配内存、跟踪其执行状态、调度 CPU 资源**，并在其完成后清理残余资源。这一动态全生命周期管理对象即为**进程（Process）**。

2. 对象模型：进程实体在内核中的数据结构

- **PCB (Process Control Block)**: OS 管理进程的核心数据结构，包含 PID、PPID、State (Running/Ready/Blocked 等)、CPU 寄存器备份、内核栈指针、页表基址 (CR3)、打开文件描述符表指针 (FD Table) 等。
- **虚拟地址空间 (Address Space)**: 由代码段 (.text)、已初始化数据段 (.data)、未初始化数据段 (.bss)、堆 (Heap)、共享库映射区、栈 (Stack) 构成的私人地址空间。

3. 状态模型：经典进程状态机



4. 不变量 (Invariants)

- 1. **Ready 唯一缺失资源为 CPU**：就绪态进程所需的物理内存、I/O 条件均已满足，只等待调度器分配 CPU；
- 2. **Blocked 即使给 CPU 也无法运行**：阻塞态进程因等待外部事件（如磁盘 I/O 完成、信号量），即使立刻给它 CPU 也无法向前推演；
- 3. **Zombie 进程残余释放约束**：进程调用 `exit()` 后，其地址空间已销毁，但 PCB 必须保留在系统表中，直到父进程调用 `wait()` 读取其退出码后方可彻底被 OS 回收。

5. 事件集合：驱动进程状态演化的核心 API 与硬件事件

- `fork()`：克隆当前进程，创建子进程 PCB 与地址空间副本；
- `execve()`：用新 ELF 文件重构当前进程的地址空间与代码；
- `scheduler_pick()`：调度器从 Ready 队列挑出进程投入 Running；
- `timer_interrupt()`：时钟中断触发时间片轮转，将 Running 转为 Ready；
- `sleep()` / `io_request()`：主动或被动等待资源，将 Running 转为 Blocked；
- `wakeup()` / `io_completion()`：外部事件完成，将 Blocked 转为 Ready；
- `exit()`：进程主动或异常终结，进入 Zombie 状态；
- `wait()`：父进程回收子进程退出状态与 PCB 空间。

6. 动态轨迹：从双击启动到进程销毁的完整生命周期表格

阶段	执行动作/API	进程状态	关键数据结构变化	触发事件	内核与硬件细节动作
1	Shell <code>fork()</code>	New → Ready	分配 PID，复制父进程 PCB、页表与文件描述符表	用户回车/启动程序	采用写时复制 (COW)，子进程与父进程共享物理页框，页表项设为 Read-Only。
2	Shell <code>execve()</code>	Ready Ready	→ 释放旧地址空间，解析 ELF，建立新代码/数据/栈映射	加载二进制程序	进程 PID 不变！重置通用寄存器，将入口点 (Entry Point) 写入内核栈 TrapFrame EIP。
3	调度选中	Ready → Running	加载当前进程 CPU，恢复 TrapFrame 寄存器	CR3 到 <code>schedule()</code>	执行 <code>iret</code> ，CPU 切到 User Mode，开始从新程序 <code>main()</code> 入口执行。

4	时钟中断	Running Ready	→ 当前进程 PCB 状态写为 <code>TimerInterrupt Ready</code> , 插入 Ready 队列	时钟中断剥夺 CPU 控制权, 引发 <code>Context Switch</code> 切换其他 Ready 进程 (抢占)。
5	请求磁盘读	Running Blocked	→ 当前进程 PCB 状态写为 发起 <code>read()</code> Blocked, 插入 Wait 队列	进程放弃 CPU, OS 启动磁盘 DMA 传输, 调度器选择其他 Ready 进程运行。
6	DMA 完成	Blocked Ready	→ PCB 状态改为 Ready, 从 硬件 Disk 中断 Wait 队列移入 Ready 队列	注意: 磁盘中断发生时, CPU 正在运行进程 X! 中断程序仅将目标进程改为 Ready, 不保证立刻抢占!
7	进程退出	Running Zombie	→ 释放地址空间与页表, 保 执行 <code>exit()</code> 留 PCB 与 <code>exit_code</code>	进程大部分资源已释放, 但 PID 与 <code>exit_code</code> 留在 PCB 中等待父进程回收。
8	父进程回收	Zombie → Terminated	→ 从内核 Task Array 中彻底删除 PCB, 回收 PID	父进程拿到 <code>exit_code</code> , PCB 内存被彻底释放回到内核堆。

7. 分支路径：进程生命周期的三大典型分支

- 路径 A (写时复制 COW): `fork()` 时仅复制页表 → 任一方写入时硬件 Page Fault → 内核分配新物理页框。
- 路径 B (替换不换壳): `execve()` 保持 PID 与打开文件描述符 → 彻底覆写代码段与数据段。
- 路径 C (孤儿 vs 僵尸):
 - **Zombie (僵尸)**: 子进程先退出, 父进程未调用 `wait()` → 维持 PCB 占用 PID。
 - **Orphan (孤儿)**: 父进程先退出, 子进程存活 → 自动由 1 号 `init` 进程领养。

8. 机制与策略：进程调度与 Sleep/Wakeup 机制

408 核心机制：Blocked → Ready 与 Ready → Running 的彻底解耦

在 OS 中，`sleep(channel)` 与 `wakeup(channel)` 是底层原子机制：

- 1. **Sleep(channel)**：进程主动将自己状态设为 Blocked，挂入 `channel` 关联的等待队列，并主动调用 `schedule()` 放弃 CPU。
- 2. **Wakeup(channel)**：硬件中断（如 Disk 完成）触发内核唤醒函数，扫描等待队列，将阻塞在该 `channel` 上的进程状态改为 **Ready** 并移入就绪队列。
- 3. **关键结论：Wakeup 不等于立刻拿到 CPU 运行！** 它只是获得了重新参与 CPU 竞聘的资格（Ready）。当前 CPU 是否立刻切换，完全由调度策略（抢占式 vs 非抢占式、优先级）决定。

9. 408 题型映射

408 训练：进程生命周期高频考点

- 1. **进程状态转换推断**：必须精准区分 `Running → Ready`（时间片到/抢占）、`Running → Blocked`（等待资源/主动 `sleep`）、`Blocked → Ready`（资源就绪/中断唤醒）。
- 2. **fork() 动态返回值计算**：`fork()` 在父进程中返回子进程 PID，在子进程中返回 0；利用循环与 `fork()` 考查生成进程总数（如 n 次 `fork()` 产生 $2^n - 1$ 个子进程）。
- 3. **execve 与 PID 关系**：`execve` 成功执行后 PID 不变，代码段与数据段更新，未设置 `FD_CLOEXEC` 的打开文件保持打开。

10. 诊断与错因矩阵

学生典型错误表象	底层认知缺口诊断	正确推演矫正句
以为 I/O 中断到来后，请求 I/O 的进程立刻变成 Running 运行	混淆“事件完成 (Wakeup)”与“CPU 调度 (Schedule)”	I/O 中断只负责将进程从 Blocked 转为 Ready，之后必须等待调度器 pick 才能变为 Running。
混淆 fork() 与 execve() 的功能	混淆“创建进程容器”与“重载程序内容”	fork() 创建一个与父进程一模一样的副本（新 PID）；execve() 保持原 PID 不变，替换地址空间为新 ELF 程序。
认为进程退出后其 PCB 立即被销毁	未理解 Zombie 状态与父子进程收尸机制	进程退出后变为 Zombie，PCB 与 Exit-Code 仍保留在系统表中，只有父进程执行 wait() 后 PCB 才彻底销毁。

4.4 篇 II：并发——所有允许的交错都必须正确

并发的数学对象

并发程序不是一条执行轨迹，而是一组可能的 *interleavings*。正确性意味着：对于所有允许的交错，程序不变量都不能被破坏。

一条高级语句如 `counter++` 可能展开为 `load/add/store`；上下文切换可以发生在底层步骤之间，因此“源码看起来一行”不意味着原子。

4.4.1 同步只有两类基本任务

1. **Mutual Exclusion / Atomicity**：哪些操作之间不允许其他线程插入？
2. **Condition / Ordering**：哪个条件成立后才能继续？哪个事件必须先发生？

先判断约束类型，再选 `lock`、`condition variable`、`semaphore`、`join` 等工具。

4.4.2 锁：不是布尔值，而是一个小型调度系统

高质量锁需要考虑：锁状态、原子状态转换、等待队列、唤醒、公平和性能。硬件原子指令（`test-and-set`、`CAS`、`LL/SC`、`fetch-and-add` 等）解决“原子争用”问题，但等待是否自旋、`yield` 还是睡眠需要 OS 支持和策略。

4.4.3 条件变量：等待的是状态谓词，不是 `signal`

经典模式：

```
lock(m);
while (!predicate)
    wait(cv, m);
use_state();
unlock(m);
```

条件变量只是等待队列/通知机制，真正的“条件”是共享状态上的布尔谓词。被唤醒只表示状态可能变化，因此醒来必须重新检查。

4.4.4 信号量：整数代表“许可数量”

初值 1 可表示一个互斥许可；初值 0 可表示事件尚未发生；初值 N 可表示 N 个同类资源。信号量题先问“整数的现实语义”，再写 P/V，而不是背模板。

4.4.5 并发错误与进展

非死锁错误可优先检查：

- **Atomicity violation**：本应整体完成的多步操作被插入；
- **Order violation**：程序需要 A before B，但没有机制保证。

进展性问题必须区分 Deadlock、Livelock、Starvation。死锁经典四条件是互斥、请求并保持、不可剥夺、循环等待；破坏任一必要条件可阻止死锁形成。资源分配图适合显式表示“谁持有什么、谁等待什么”。银行家算法则不是在判断“现在是否已经死锁”，而是在试探当前分配后是否仍存在一条让所有进程按最大需求完成的安全序列：Unsafe 表示未来无法保证安全，并不等于此刻已经 Deadlocked。

4.4.6 Safety / Liveness / Fairness / Performance

互斥正确并不保证公平；无死锁不保证无饥饿；正确数据结构不保证可扩展。并发性能常见根因不是“锁本身慢”，而是所有核心是否争同一个共享热点。分桶、分区、per-CPU 数据、局部累积、缩短临界区等都在降低共享。

4.4.7 经典同步模型的统一解释

生产者—消费者、读者—写者、哲学家、理发师、屏障、线程池、有界队列都应先画：

共享状态 + 容量/资源 + 顺序约束 + 互斥约束。

然后再决定同步工具。

4.4.8 事件驱动：并发不只有多线程

单线程 event loop 可按事件逐个推进状态，从而减少共享内存竞态；代价是处理函数不能长时间阻塞，需要非阻塞/异步 I/O，把长期操作拆成显式状态机、回调或 continuation。复杂性不会消失，只是从“隐式线程交错”转移成“显式状态管理”。

4.4.9 Priority Inversion：优先级与资源依赖会相互作用

高优先级线程若等待低优先级线程持有的锁，而中优先级线程不断抢占低优先级线程，就会出现优先级反转。它提醒我们：调度优先级与同步资源不能独立分析，实时系统常需要 priority inheritance 等机制控制这种依赖。

408 训练：并发题七问

1. 哪些状态共享，哪些私有？
2. 哪些源码操作其实由多步底层动作构成？
3. 必须保持什么 invariant？
4. 哪种交错会破坏它？
5. 需要互斥、顺序、资源计数还是容量限制？
6. 等待应自旋还是睡眠？
7. 是否存在 deadlock/livelock/starvation 或严重竞争？

4.5 篇 III：内存虚拟化——一块物理内存怎样像很多私人地址空间

4.5.1 核心目标：透明、效率、保护

程序看到连续虚拟地址空间；底层物理页框可以离散、共享、按需分配甚至临时不驻留。VM 首先是地址空间抽象与保护机制，不是简单“硬盘扩大内存”。

4.5.2 连续分配、分段、分页：三种不同的“空间切法”

连续分配直接给进程一段连续物理区，地址转换简单，但容易受到外部碎片和动态增长限制；分段按程序逻辑区域组织，便于保护/共享但段长可变；分页把虚拟与物理空间都切成固定大小块，用页表解除连续性要求，代价是页表、内部碎片与地址翻译开销。段页式则把逻辑分段与固定页结合。不要只背优缺点，应问：它选择固定还是可变粒度，换来了什么映射自由度和碎片类型？

4.5.3 间接映射

$$Virtual\ Address \rightarrow Page\ Table/TLB \rightarrow Physical\ Address$$

页面机制把地址拆为 VPN + Offset，转换只改变页号映射，页内偏移保持。多级页表用稀疏间接结构节省页表空间。

页表项不只是“地址转换”：还承载 present/valid、读写、用户/内核、dirty、accessed 等映射、保护和状态信息。

4.5.4 TLB Miss、Page Fault、Cache Miss 三分

概念	核心特征与机制区别
TLB Miss	• 地址翻译缓存未命中，页面可能已在主存中 • 可由硬件或软件页表查询填充 TLB
Page Fault	• 缺页异常，当前访问触发 OS 内核处理机制 • 包含页面未驻留 Disk I/O、COW 复制或权限越界异常
Cache Miss	• CPU 指令/数据缓存未命中 • 属于存储层次结构访问慢路径，与 TLB 独立

4.5.5 快路径与异常慢路径

$$TLB\ Hit \rightarrow TLB\ Miss + PageTable \rightarrow PageFault + I/O.$$

正常路径硬件完成，异常路径 trap 到内核，可能分配 frame、从 backing store 读页、更新 PTE/TLB，最后重启原指令。

4.5.6 页面置换与局部性

请求分页机制解决“可以按需装入”；FIFO/LRU/CLOCK/OPT 等解决“内存不够时牺牲谁”。LRU/working set 的合理性来自局部性。Thrashing 表示工作集总需求超过物理能力，大量时间浪费在换页而非有用执行。

4.5.7 延迟执行思想

Demand paging、lazy allocation、copy-on-write、write-back 都在做：先不付成本，真正需要时再付。它降低常见路径成本，但把第一次访问推入慢路径。

4.6 篇 IV：I/O——同步 CPU 怎样接入异步慢设备

I/O 的统一模型：

$$Fast\ Synchronous\ CPU \leftrightarrow Slow/Asynchronous\ Device$$

三次“卸载”：

- Polling → Interrupt：卸载等待；
- PIO → DMA：卸载搬运；
- Device-specific logic → Driver：卸载设备知识。

DMA 的典型路径：CPU 配置地址/长度 → DMA 申请总线/内存访问 → 设备与主存间批量传输 → 完成或出错中断 CPU。CPU 不是完全不参与，而是不再逐字节搬运。

4.6.1 Buffer / Cache / Spooling 不同目的

机制/抽象	核心引入目的与主要解决矛盾
Buffer	• 吸收生产者与消费者之间的速度或数据粒度差异，平滑数据流
Cache	• 保存未来可能被高频复用的数据副本，隐藏慢速存储延迟
Spooling	• 用队列与辅助存储将独占设备包装成并发共享的逻辑服务
DMA	• 批量数据传输，降低 CPU 逐字节数据搬运负担
Interrupt	• 异步事件通知，降低 CPU 主动轮询等待负担

4.6.2 磁盘调度必须先有设备成本模型

HDD 访问时间常分：

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}.$$

SSTF、SCAN 等策略的意义来自寻道/旋转成本；设备换成 SSD 后传统磁头移动模型的重要性会下降。策略必须建立在物理成本模型上。

4.6.3 经典磁盘调度策略的目标差异

FCFS 强调到达顺序与简单公平；SSTF 优先当前磁头最近请求，平均寻道可能更小但可能使远端请求饥饿；SCAN/C-SCAN 用有方向的“电梯”扫描改善全局公平与可预测性；LOOK/C-LOOK 只扫到当前最远请求。做题时先画磁道位置和服务方向，再逐步更新磁头位置，不要在脑内跳步。

4.7 篇 V：文件与持久化——从磁盘块到可命名对象

4.7.1 名称不是对象

Unix 风格文件系统的核心关系可以画成：

$$\text{Name} \rightarrow \text{DirectoryEntry} \rightarrow \text{inode} \rightarrow \text{DataBlocks}$$

而进程访问打开文件还多一层：

$$fd \rightarrow \text{OpenFileObject} \rightarrow \text{inode}.$$

这解释了硬链接、unlink、打开引用、文件偏移等：一个 inode 可以有多个名字；删除目录项不等价于立即销毁仍被引用的对象。

4.7.2 文件系统是一组持久化映射

$$\text{Name} \rightarrow \text{Object} \rightarrow \text{LogicalBlocks} \rightarrow \text{DeviceBlocks}$$

盘上结构典型包括 superblock、inode/data bitmap、inode table、directory data、direct/indirect blocks、data region。做题要同时看“盘上结构”与“访问方法”。

4.7.3 一次系统调用要展开成访问路径

例如：

$$\text{open}("/a/b/c") \rightarrow \text{pathname traversal} \rightarrow \text{inode lookup} \rightarrow \text{open file state}.$$

$$\text{read}(fd) \rightarrow fd \text{ table} \rightarrow \text{file object} \rightarrow \text{inode} \rightarrow \text{block mapping} \rightarrow \text{page cache} \rightarrow \text{device}.$$

“一个高层调用”往往对应多次元数据和数据访问。

教材模型 vs 现代工程：VFS

现代 Linux 用 VFS 为用户态提供统一文件系统接口，并允许不同文件系统共存。路径查找还利用 dentry cache 加速 pathname 到 dentry/inode 的映射。408 不要求记 Linux 内核结构体，但这个实例很好地说明“统一接口 + 间接层 + 缓存”的系统思想。

4.7.4 文件访问、分配与空闲空间：三套问题不能混

- 访问方法：顺序访问、随机/直接访问等回答“应用怎样定位文件内容”；
- 分配方法：连续、链接、索引等回答“文件逻辑块怎样映射到设备块”；

- 空闲空间管理：bitmap、空闲链表等回答“哪些块还可分配”。

连续分配局部性好但扩展困难；链接分配扩展灵活但随机访问差；索引分配用额外索引结构换来随机访问与灵活扩展。inode 的 direct/indirect blocks 是索引式思想的经典实例。

4.7.5 持久化不是“调用 write 就结束”

高层更新可能对应 inode、bitmap、data、directory 等多次物理写。系统可能在任意两次稳定写之间崩溃，因此必须规定合法写顺序、提交点和恢复方式。fsck 是事后扫描修复；journaling/WAL 则先记录更新意图/内容，commit 后才能把事务视为可重做的逻辑整体。

崩溃一致性统一分析法

列出所有底层写 → 列出必须满足的先后关系 → 标记 commit point → 枚举每个 crash point → 判断忽略、redo、扫描修复或是否产生非法状态。

4.8 OS 的研究级扩展：保留思想，不挤占 408 主线

以下内容来自用户原始 OS 方法论材料中的持久化扩展，虽然部分超出 408 核心考纲，但对形成完整系统思想很有价值：

- RAID：在容量、可靠性、性能之间权衡；RAID0 条带无冗余，RAID1 镜像，RAID4/5 用校验，性能必须结合随机/顺序、读/写、请求大小与吞吐/延迟分析。
- FFS/Block Group：把相关 inode/目录/数据放近，体现“布局服从访问局部性”。
- LFS：把随机更新转成顺序追加，以前台快速写入换后台 cleaning；类似思想可迁移到 LSM、GC、SSD FTL。
- SSD/FTL：逻辑块 → 物理 Flash 映射；页编程、块擦除、磨损限制导致异地更新、垃圾回收和磨损均衡。
- 数据完整性：冗余负责能否重建，checksum 负责能否发现异常，backup 负责恢复历史版本；三者不能混淆。
- 分布式持久化：请求/回复可能丢失，服务器可能崩溃；幂等操作使超时重试更安全；客户端缓存提高扩展性但引入一致性语义。

4.9 Protection：不是一章，而是横向约束

CPU 的 user/kernel mode、页表权限、文件权限、特权指令、进程隔离都在回答：

Who can do What to Which Object?

每次资源访问都应检查对象是否存在、主体是否有权限、访问类型是否合法、是否越界、共享后是否需要同步。保护不是虚拟化的附属品，而是多用户/多进程系统能成立的前提。

4.10 十二个 OS 横向母思想

1. 资源 抽象；
2. 时间复用 空间复用；
3. 机制 策略；
4. 状态 事件；
5. 映射 间接层；
6. 控制 特权；
7. 共享 隔离；
8. 快路径 慢路径；
9. 立即 延迟；
10. 缓存 一致性；
11. Safety Liveness Fairness；
12. Failure Recovery。

4.11 超级母题：一次 read() 的一生

假设用户进程执行 `read(fd, buf, 4096)`，目标数据不在 page cache：

User Mode → syscall/trap → Kernel 检查 fd 与 buf 权限 → fd→open-file→inode → file offset→block mapping → page cache miss → 发起设备 I/O → 当前进程 Running→Blocked → Scheduler 选择其他 Ready 进程并 Context Switch → DMA 搬运 → Device Interrupt → 内核完成 I/O/更新缓存 → 原进程 Blocked→Ready → 未来再次被调度 → 数据复制/映射给用户 → return-from-trap → User Mode

这一条路径同时串起：系统调用、特权、进程状态、调度、文件映射、缓存、I/O、DMA、中断、虚拟内存和恢复。

4.12 第二母题：一个进程的一生

fork/创建 → 资源与执行上下文 → exec/建立新地址空间 → demand paging/page fault → timer interrupt/抢占 → scheduling/context switch → lock/wait/I/O blocking → interrupt wakeup → 文件访问 → exit → 引用与资源回收

4.13 408 OS 统一解题协议

408 训练：OS 九问

1. 真实稀缺资源是什么？
2. 上层操作的是哪个抽象？
3. 当前对象与状态分别是什么？
4. 什么事件触发状态变化？
5. 当前在用户态还是内核态，硬件和 OS 谁负责什么？
6. 中间有哪些映射/间接层？偏移或身份哪些保持？
7. 当前走 fast path 还是 fault/block/I/O slow path？
8. 这是 mechanism 还是 policy？必须保持什么 invariant？
9. 最终状态和时间/空间/I/O/公平代价是什么？

OS 高频混淆

Ready $\neq$ Blocked；mode switch $\neq$ context switch；进程 $\neq$ 程序；线程共享地址空间 $\neq$ 所有状态共享；mutex/lock $\neq$ condition synchronization；signal $\neq$ condition true；unsafe state $\neq$ 已经 deadlocked；virtual address $\neq$ physical address；TLB miss $\neq$ page fault $\neq$ cache miss；buffer $\neq$ cache $\neq$ spooling；write() 返回 $\neq$ 数据必然已到稳定介质；文件名 $\neq$ inode $\neq$ open file object。

三科统一：真正的 408 系统方法论

5.1 同一个思想在三门课里的不同化身

母思想	网络 (Net)	计组 (CO)	操作系统 (OS)
抽象 / 分层	• 五层体系与协议接口	• ISA / 微架构、存储层次	• 进程、虚存、文件、VFS
状态机	• TCP / DHCP / 路由状态	• 指令状态转移、流水线	• 进程 / 页 / 锁 / 文件状态
映射	• DNS、路由表、ARP	• 地址字段、Cache 映射	• VA→PA、fd→file→inode
局部性 / 缓存	• DNS 缓存、网络 CDN	• Cache、TLB 概念边界	• TLB、Page Cache、dentry
资源共享	• 链路统计复用	• Bus / ALU / Memory 端口	• CPU / 内存 / 锁 / 设备
反馈	• ACK、超时、拥塞通知	• 分支预测、内存响应	• 中断、唤醒、缺页重试
快慢路径	• 正常发送 vs 异常重传	• Cache hit/miss、Branch hit/miss	• TLB hit vs Page Fault
保护 / 边界	• 端到端作用域、协议边界	• ISA / 特权级边界	• User/Kernel、页与文件权限
性能	• 带宽 / RTT / 队列延迟	• IC / CPI / 时钟周期	• 调度 / 等待 / 缺页 I/O
故障恢复	• 丢包重传 / 路由收敛	• 异常响应 / 状态恢复	• 崩溃一致性 / 死锁恢复

5.2 一条“程序读取网络数据”的跨三科路径

考虑应用执行 `recv/read(socket, ...)`:

1. 网络侧：远端数据经历应用/运输/IP/链路/物理封装与逐跳转发；
2. 网卡收到帧并通过 DMA 把数据搬到主存，产生中断；
3. 计组侧：CPU 响应中断，执行内核指令，访问 Cache/TLB/DRAM；

4. OS 侧：驱动与协议栈处理数据，唤醒阻塞在 socket 上的进程；
5. 调度器未来把 CPU 分给该进程；
6. 用户态读取返回，应用继续运行。

这说明 408 三门课不是三套世界：网络数据最终必须由真实硬件执行，并由 OS 管理其资源与控制流。

5.3 真正的综合题解题顺序

遇到跨章节题，建议固定按照：

对象 → 层次/作用域 → 当前状态 → 事件 → 映射路径 → 资源竞争 → 状态更新 → 代价

5.3.1 第一步：先定边界，不急着算

问清：现在讨论的是应用语义、OS 抽象、ISA 可见状态还是硬件内部状态？如果层次没定，公式越算越乱。

5.3.2 第二步：画动态轨迹

网络题画 packet/time line；计组画 cycle/datapath；OS 画 process/resource/state table。复杂系统题最怕只看静态定义。

5.3.3 第三步：明确“不变量”

TCP 不能把已确认之前的数据当未确认；流水线优化不能破坏 ISA 语义；并发不能破坏共享状态 invariant；文件系统恢复后必须落在允许状态集合。

5.3.4 第四步：最后才进入数字计算

位数、地址、时延、CPI、页框、P/V 值都是最后一步。先有模型，数字只是执行。

5.4 三科的训练记录模板

5.4.1 网络题记录

- 作用域；
- 谁保存状态；
- 触发事件；
- 字段谁读/谁改；
- 正常路径与异常路径；
- 最终性能量及单位。

5.4.2 计组题记录

- 当前体系结构状态；
- 数据位置；
- 数据通路；
- 控制与资源冲突；
- cycle/critical path；
- 最终提交状态与性能代价。

5.4.3 OS 题记录

- resource / abstraction；
- state / event；
- user/kernel 与 hardware/kernel 分工；
- mapping / data structures；
- mechanism / policy；
- invariant / wait / fault / recovery；
- 时间、空间与 I/O 代价。

完整查漏清单：防止“理解很高，但考试漏点”

6.1 计算机网络覆盖清单

- 性能: rate、bandwidth、throughput、transmission/propagation/queueing/processing delay、RTT、BDP、utilization。
- OSI/TCP-IP/五层分析模型的对应关系、PDU、encapsulation/decapsulation。
- 应用: DNS、HTTP/版本、WWW、C/S、P2P、FTP、SMTP/POP3/IMAP。
- 运输: port/socket、mux/demux、TCP 4-tuple/5-tuple、UDP、TCP 字节流、握手/关闭/TIME_WAIT、Seq/ACK、累计确认、快速重传、SACK、滑窗、flow/congestion control。
- 网络: IP best effort、forwarding/routing、gateway、IPv4 header/fragmentation、CIDR/subnet/aggregation/LPM、DHCP、ICMP、NAT/NAPT、IPv6、DV/LS、RIP/OSPF/BGP、AS、multicast、Mobile IP。
- 链路: framing、transparent transmission、HDLC stuffing、parity/CRC/Hamming、PPP、MAC、多路复用、ALOHA/CSMA、CSMA/CD、CSMA/CA、MAC address、ARP/NDP、hub/bridge/switch、self-learning/flooding、collision/broadcast domain、VLAN。
- 物理: source/sink/signal/channel、bit/symbol rate、encoding/modulation、Nyquist/Shannon/SNR、media、repeater/hub、circuit/message/packet switching、datagram/virtual circuit。

6.2 计算机组成原理覆盖清单

- 系统概述与性能: clock、IC、CPI、IPC、MIPS、CPU time；
- 指令/ISA: opcode、operand/address fields、machine instruction、assembly/mnemonic、instruction types、RISC/CISC、x86/MIPS/ARM/RISC-V；
- 数据表示与运算: bit width、unsigned/signed、sign-magnitude/ones' complement/two's complement/biased、modular arithmetic、定点乘除与移位、overflow、IEEE 754 浮点、ALU；
- 存储: register/cache/DRAM/SRAM/ROM/secondary storage、memory chip 位/字扩展、bank/interleaving、hierarchy、locality、mapping/replacement/write policy、coherence 概念；
- 冯·诺依曼与 Harvard/modified Harvard 边界；
- 指令周期与微操作: fetch/indirect/execute/interrupt 以及 IF/ID/EX/MEM/WB; PC/IR/GPR/ALU/control/...

- hardwired vs microprogrammed control;
- single-bus、single-cycle、multi-cycle、pipeline; 三类 hazard 与 forwarding/stall/prediction;
- bus: data/address/control、arbitration/timing/synchronization;
- I/O: polling、interrupt、DMA、设备异步性;
- superscalar、superpipelining、VLIW、multicore/multiprocessor。

6.3 操作系统覆盖清单

- 总纲: resource/abstraction、time/space multiplexing、mechanism/policy、state/event、indirection/mapping、fast/slow path、locality/cache、lazy/demand、isolation/protection。
- 控制与特权: user/kernel、system call、trap/exception/interrupt、mode/context switch、timer。
- Process/Thread: 状态、PCB/执行上下文概念、创建/终止/阻塞/唤醒、thread shared/private state、IPC (pipe/shared memory/message/notification)。
- Scheduling: FCFS/SJF/SRTF/RR/Priority 等经典模型, 指标与 starvation/fairness。
- Concurrency: race/critical section/atomicity/order、hardware atomic primitives、locks、condition variables、semaphores、经典同步模型、event-driven model、deadlock four conditions、banker/safe state、livelock/starvation/priority inversion、lock granularity/scalability。
- Memory: contiguous allocation、segmentation/paging/segment-paging、fragmentation、page table/multi-level、TLB、VM/demand paging、page fault、replacement FIFO/LRU/CLOCK/OPT、working set/thrashing、COW/lazy allocation、protection/sharing。
- I/O: device interface/driver、polling/interrupt/DMA、buffer/cache/spooling、disk physical cost、disk scheduling。
- File: file/directory/path/fd/open-file/inode、hard/symbolic link、access methods、contiguous/linked/indexed allocation、bitmap/free-list、direct/indirect mapping、path lookup、VFS 思想。
- Persistence extension: RAID、FFS locality、journaling/crash consistency、LFS、SSD/FTL、checksum vs redundancy vs backup、distributed retry/idempotence/cache consistency。

参考资料与边界说明

7.1 本手册的资料层次

1. 第一层：用户提供材料与本次对话中的三科总结。主体结构、考试重点和大量细节来自这些材料；
2. 第二层：经典教学框架。OS 主要参考 OSTEP 的 virtualization/concurrency/persistence 组织思想与 MIT xv6 的 trap/system call/page table/file system 实践视角；
3. 第三层：现代工程校准。使用 RFC、RISC-V 官方规范和 Linux Kernel Documentation 校正 HTTP/3、IPv6、ISA/privileged architecture、VFS、现代调度器等容易被绝对化的陈述。

7.2 推荐外部资料

- Operating Systems: Three Easy Pieces: pages.cs.wisc.edu/~remzi/OSTEP/
- MIT xv6 / 6.1810: pdos.csail.mit.edu/6.S081/2024/xv6.html
- RISC-V Ratified Specifications: docs.riscv.org/reference/home/index.html
- RFC 9000 QUIC: rfc-editor.org/rfc/rfc9000.html
- RFC 9114 HTTP/3: rfc-editor.org/rfc/rfc9114.html
- RFC 8200 IPv6: rfc-editor.org/rfc/rfc8200.html
- Linux Kernel Documentation: docs.kernel.org/

7.3 最后的学习原则

不要背现象，要寻找生成现象的机制

网络不要只背协议字段，要知道作用域、状态、反馈和资源竞争；计组不要只背框图，要知道数据在哪里、经过什么路径、在哪个时钟边界提交；OS 不要只背状态和算法，要知道真实资源、抽象、事件、机制、策略和必须维持的不变量。

最终目标不是“见过这道题”，而是“没见过，也知道系统为什么只能这样演化”。